

Теория Параллелизма

Отчет

Уравнение теплопроводности на OpenACC

Выполнил 22930 Никитин И.В.

06.07.2024

Цель работы: реализовать решение уравнение теплопроводности в двумерной области на равномерных сетках, перенести программу на GPU используя директивы *OpenACC*. Сравнить скорость работы для разных размеров сеток на центральном и графическом процессорах.

Используемый компилятор: pgc++.

Используемый профилировщик: NSightSystems (NVIDIA).

Как проводили замер работы: с помощью библиотеки “std::chrono”.

Выполнение на CPU

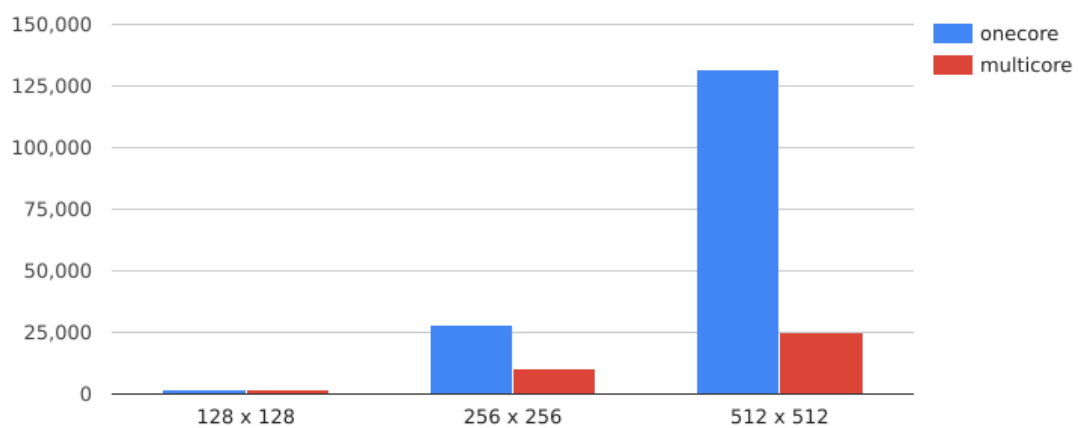
CPU-onecore

Размер сетки	Время выполнения (ms)	Точность	Количество итераций
128x128	1919	1-6e	27900
256x256	27824	1-6e	91200
512x512	131567	35-6e	100000

CPU-multicore

Размер сетки	Время выполнения (ms)	Точность	Количество итераций
128x128	1615	1-6e	27900
256x256	10459	1-6e	91200
512x512	24849	35-6e	100000
1024x1024	63179	1-6e	289800

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

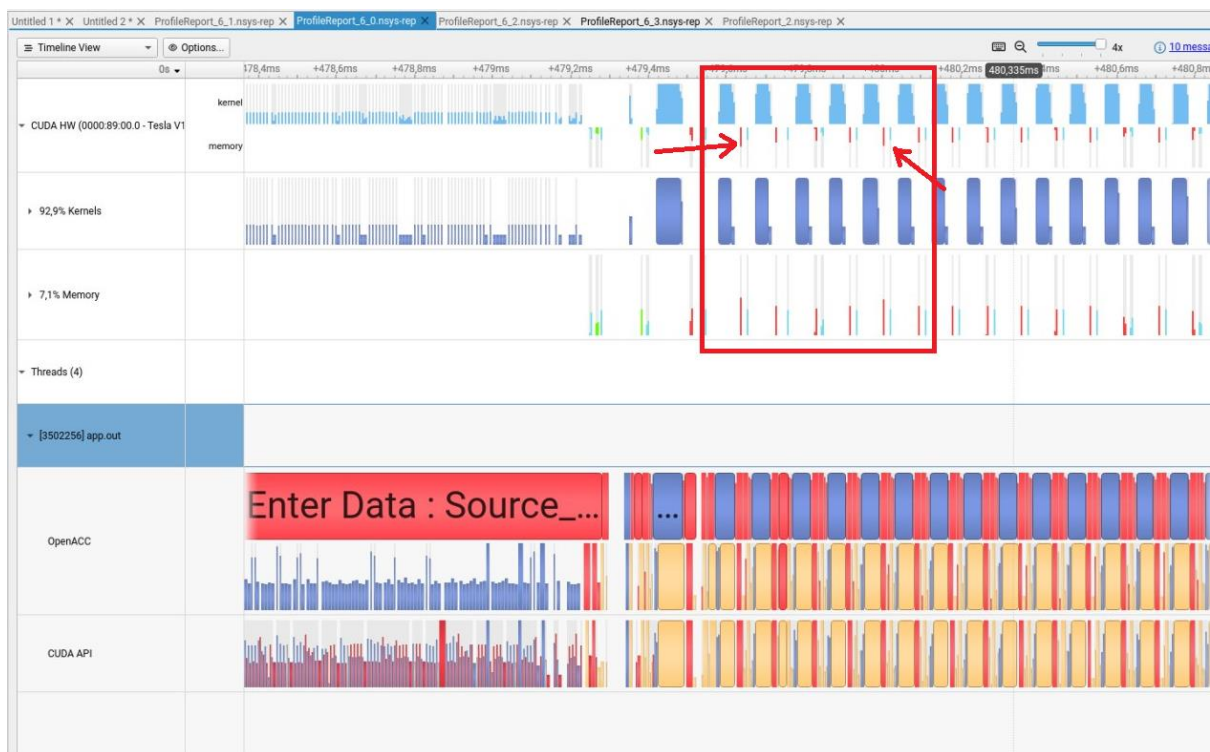
Этапы оптимизации на сетке 512 x 512

Этап №	Время выполнения (ms)	Точность	Максимальное количество итераций	Комментарии (что было сделано)
1	17940	1-6e	289578	Был реализован код с использованием более точного распараллеливания, а именно с использованием <i>parallel gang vector</i>
2	9264	1-6e	289650	Была добавлен интервал обновления ошибки через 150 итераций
3	9169	1-6e	289800	Изменение интервала обновления ошибки из 150 на 300 итераций

4	6689	1-6e	289650	Запуск вычисления сетки и обновления ошибки в асинхронно м режиме
---	------	------	--------	---

Этап 1:

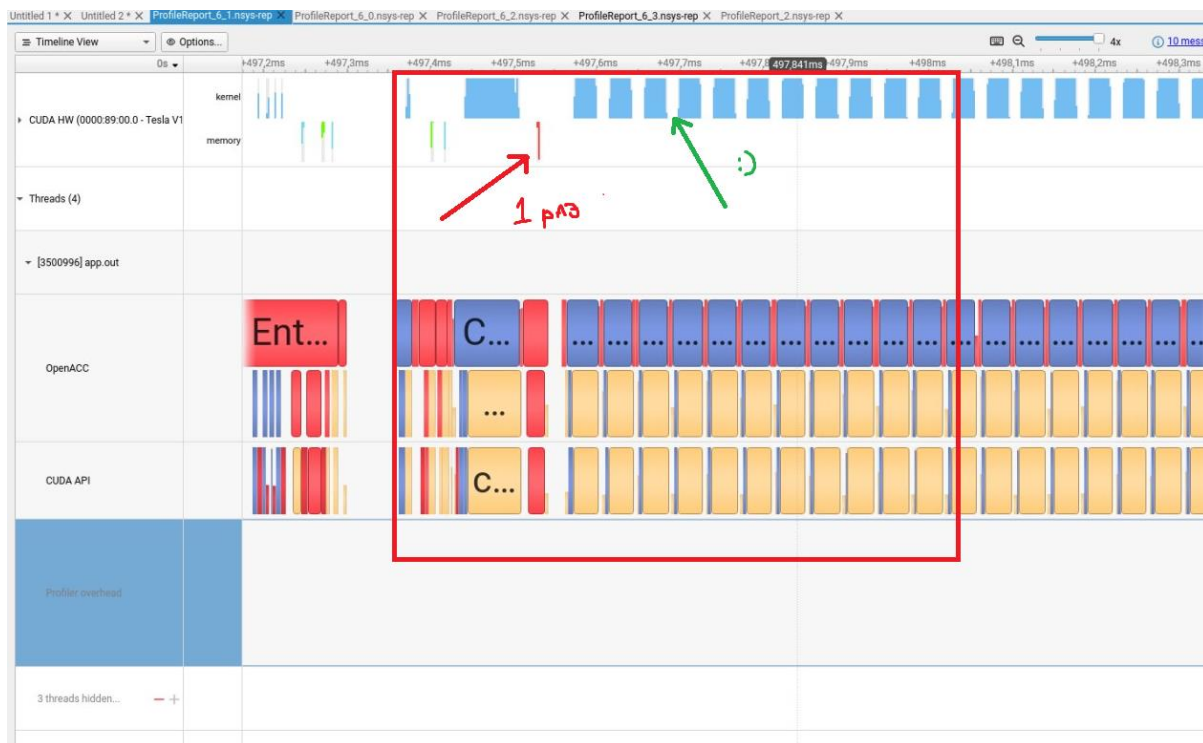
После запуска кода и последующей профилировки было выявлено сильное торможение кода в процессе простоя в моменты копирования данных между хостом и устройством.



Этап 2:

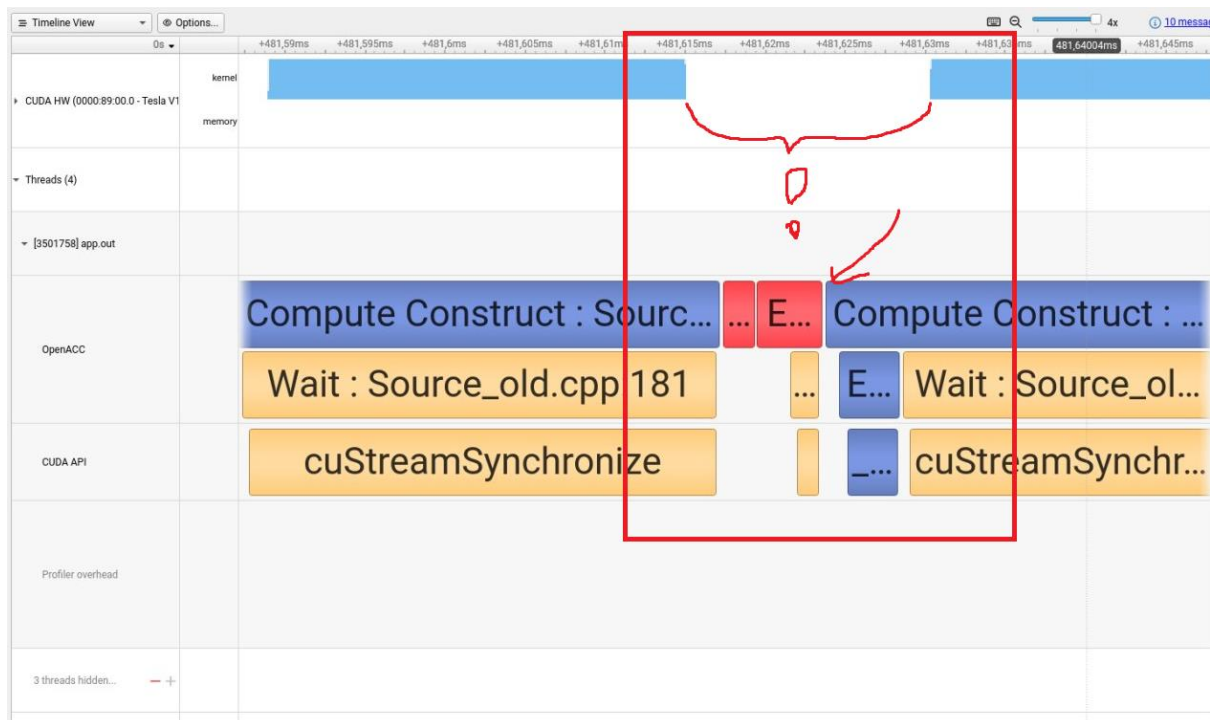
Для того чтобы исправить ситуацию из первого этапа пришлось добавить интервал обновления ошибки в 150 итераций. Число итераций было подобрано экспериментально, таким образом, программа больше тратит время на само вычисление сетки, а не на передачу данных

между хостом и устройством, которое много берет временных ресурсов.



Этап 3:

Была попытка еще сильнее сузить время выполнения путем увеличения интервала из второго этапа до 300 итераций, результат оказался не совсем впечатлительным(... Все из-за постоянного копирования данных из хоста в память устройства после каждой итерации, которые действуют как блокировочные операции.



Этап 4:

Было найдено прекрасное решение по предотвращению блокировки кода на моменты манипуляций с памятью хоста и устройства - асинхронное выполнение вычисления сетки. Такой подход практически полностью разгрузил всю нагрузку проброса памяти с потока вычисления, тем самым ускорив вычисление на максимум.

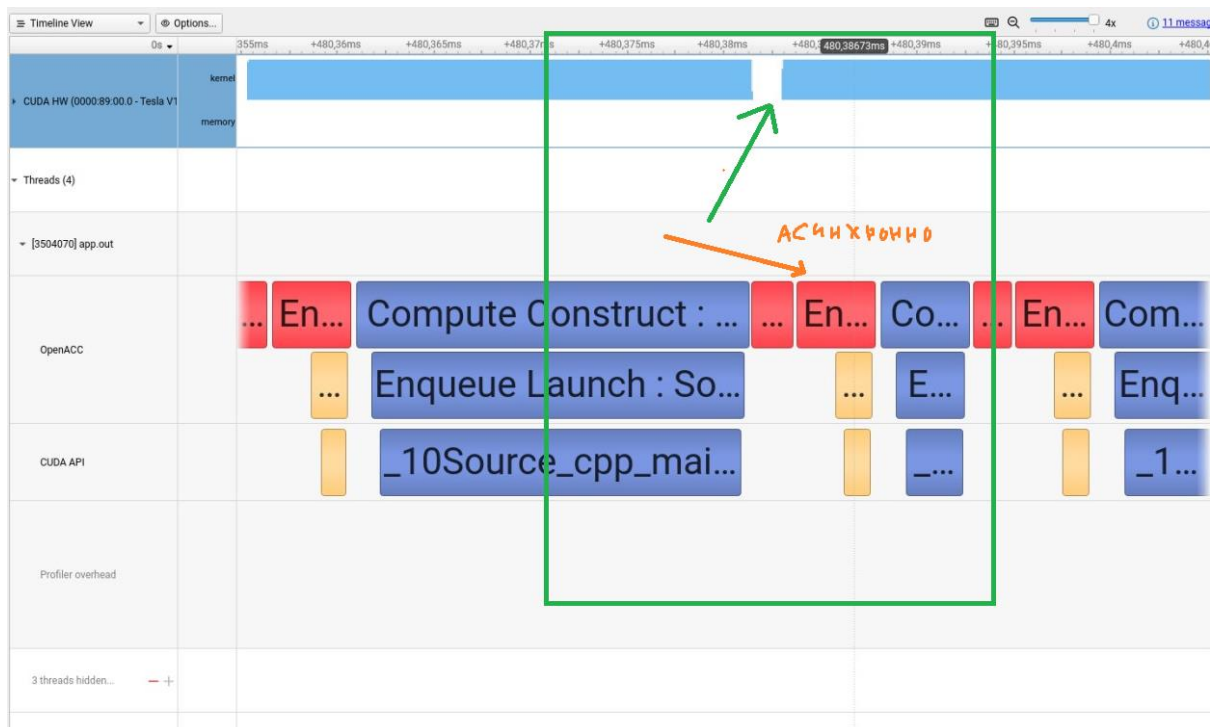
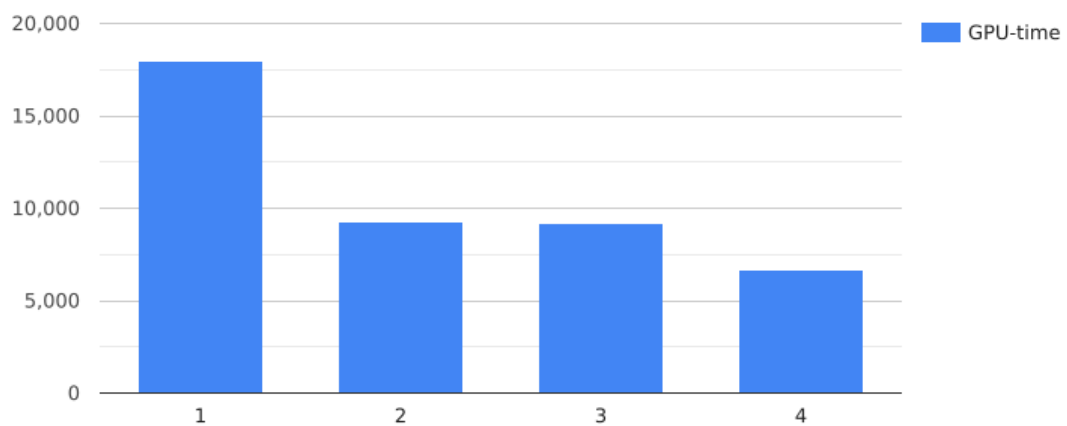


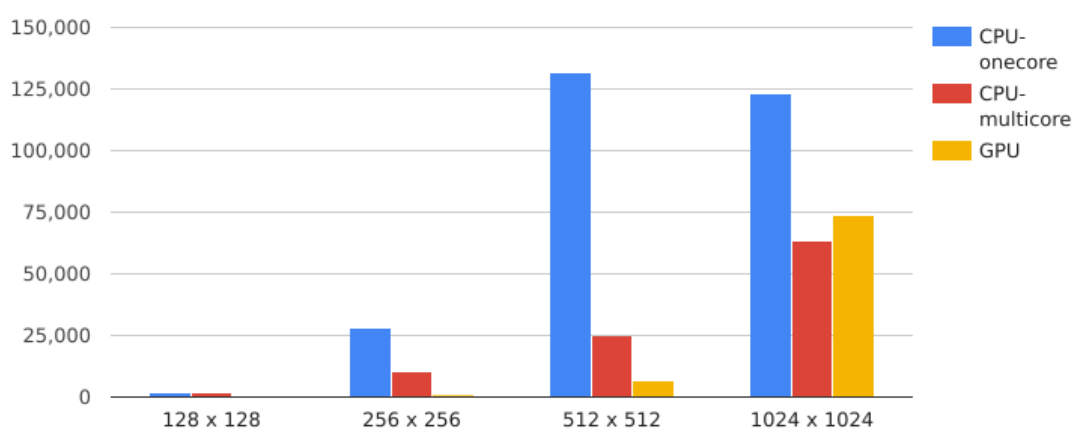
Диаграмма оптимизации



GPU – оптимизированный вариант

Размер сетки	Время выполнения (ms)	Точность	Количество итераций
128x128	308	1-6e	27750
256x256	910	1-6e	91200
512x512	6689	1-6e	289650
1024x1024	74006	1-6e	861300

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Вывод:

Исходя из выше приведенных манипуляций по оптимизации кода на OpenACC можно сказать что передача данных между памятью хоста и устройства является самой тормозящей проблемой в скорости работы программы, тем не менее, грамотный подход к решению проблемы, как с

использованием асинхронных операций может помочь значительно исправить такую проблему в работе кода.

Вот ниже предоставленный отчет сборки кода на NVIDIA OpenACC:

```
START COMPILING PROGRAM ON GPU...
main:
  106, Generating enter data copyin(bias_1)
    Generating enter data create(main_arr[:bias_1+size][:bias_1+size])
    Generating enter data copyin(size,gradient)
    Generating enter data create(main_arr2[:bias_1+size][:bias_1+size])
    Generating present(main_arr2[:][:],gradient,main_arr[:][:],size,bias_1)
    Generating NVIDIA GPU code
  109, #pragma acc loop gang(4), vector(128) /* blockIdx.x threadIdx.x */
  119, #pragma acc loop gang(4), vector(128) /* blockIdx.x threadIdx.x */
  152, Generating enter data copyin(error)
  164, Generating update device(error)
    Generating present(main_arr[:][:],main_arr2[:][:])
    Generating implicit firstprivate(size)
    Generating NVIDIA GPU code
  166, #pragma acc loop gang /* blockIdx.x */
    Generating reduction(max:error)
  169, #pragma acc loop vector(128) /* threadIdx.x */
  164, Generating implicit copy(error) [if not already present]
  181, Generating present(main_arr[:][:],main_arr2[:][:])
    Generating implicit firstprivate(size)
    Generating NVIDIA GPU code
  183, #pragma acc loop gang /* blockIdx.x */
  186, #pragma acc loop vector(128) /* threadIdx.x */
  206, Generating update self(error)
COMPILING HAS ENDED!
```

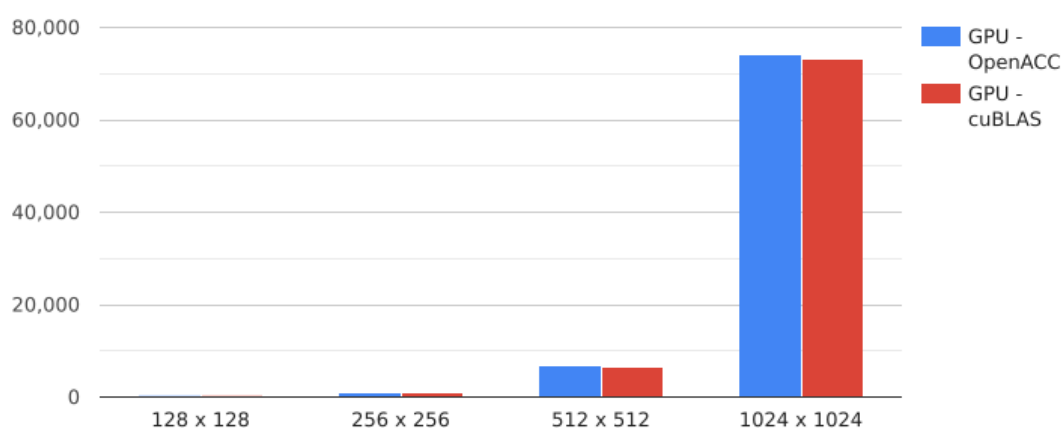
Дополнение к отчету:

Использование cuBLAS

Выполнение на GPU с использованием cuBLAS

Размер сетки	Время выполнения (ms)	Точность	Количество итераций
128x128	308	1-6e	31050
256x256	893	1-6e	104400
512x512	6551	1-6e	342000
1024x1024	73171	1-6e	1000000

Диаграмма сравнение с использованием cuBLAS и без



Вывод:

Как мы видим, использование библиотеки cuBLAS действительно является более оптимизированным вариантом работы, но результат не сильно отличается от предыдущей реализации. Это происходит потому что большая часть времени уходит на перенос данных с устройства на хост и назад, так как библиотека cuBLAS принимает данные с памяти хоста.

